Event handlers and performance

Why sharing state costs more, and how cssimpler keeps the common case fast

An event handler tells a button or another UI element what to do when an event happens. The important difference is whether the handler only needs to know **which function to run**, or also needs to carry **specific values from the app**.

The first case is cheaper to store and copy. The second lets each app instance have its own state, but it needs additional storage. The current implementation supports both, so an app pays for the additional storage only where it uses a captured handler.

Two ways to handle a click

A plain function

```
fn say_hello() {
    println!("Hello!");
}

let button = ui! {
    <button onclick={say_hello}>Say hello</button>
};
```

The button stores a **function pointer**: a small value that identifies the function to call. It does not carry a particular counter, document, or other piece of app state. Many buttons can point to the same function without needing a separate copy of that function.

A plain function can still access global state, but it does not have to. Here it simply prints a message. The difficulty arises when the function needs a particular app instance's state: the function pointer alone does not say which instance to use.

A handler that captures state

```
use std::sync::Arc;
use std::sync::atomic::{AtomicU64, Ordering};

let count = Arc::new(AtomicU64::new(0));
let button_count = Arc::clone(&count);

let button = ui! {
    <button onclick_shared={move || {
        button_count.fetch_add(1, Ordering::Relaxed);
    }}>
        Increment
    </button>
};
```

The expression `move || { ... }` is a **closure**. It carries `button_count` with it, so the handler knows which counter to update. A second app can create a different counter and its button will update that one instead.

`Arc` is a thread-safe shared owner. Cloning it does not copy the counter; it creates another reference to the same counter and updates a reference count. The atomic counter permits safe updates through shared references. Both choices are useful here because event handlers must be safe to share between threads.

Why the captured version needs more work

A function pointer only identifies a function. A captured handler must keep its captured values alive for as long as the UI might call it. In cssimpler's current implementation:

1. Every node reserves a small optional slot for shared handlers. Nodes that only use plain functions leave it empty.
2. When a node uses a captured handler, the library allocates a shared handler table for that node and allocates storage for the closure. The closure holds its captured values, such as the reference to the counter.
3. When a node or scene is cloned, the shared table is referenced rather than fully copied. Sharing it updates a reference count. When the last owner goes away, the shared storage can be released.
4. When an event occurs, the library looks up the appropriate handler and calls it. The current dispatch code borrows the handler, so dispatch itself does not need to increase and decrease the reference count.

Allocating means asking the memory allocator for storage that can live independently of the current function call. It is more work than putting a function pointer directly into the node. Reference counting also involves extra bookkeeping. These costs are especially visible when many handlers are created or UI trees are rebuilt frequently; they do not imply that every rendered frame becomes slower.

> **Global state is a separate choice.** An immutable global lookup table or cache can be appropriate. Mutable state that belongs to an app instance is usually easier to reason about when that instance owns it. A captured handler gives the button access to that state without requiring a global counter.

What changed in cssimpler

There are three relevant versions of the design:

- **`main`:** event handlers are plain function pointers. They are compact, but cannot capture app-specific values.
- **First closure implementation:** every handler slot stores a shared closure-capable reference, even for an ordinary function. This made captured handlers possible, but enlarged the handler storage on every node.
- **Current implementation:** plain functions remain inline. Captured handlers use an optional shared table, allocated only on nodes that need one.

The current syntax makes that choice explicit:

```
// A plain function: no captured state.
<button onclick={say_hello}>Say hello</button>

// A closure: carries the state it needs.
<button onclick_shared={move || { /* use captured state */ }}>
    Increment
</button>
```

The same distinction applies to text input: `oninput` accepts a plain function and `oninput_shared` accepts a captured handler. The corresponding Rust builder methods use names such as `.on_click(...)` and `.on_click_shared(...)`.

What the measurements showed

The measurements below were made with release builds on an Apple M1 Pro. The synthetic tree tests built and copied 1,000 nodes; reported times are medians from repeated runs. They show the cost of the design in that test, rather than predicting every application's performance.

Measurement	`main`	Current
Handler storage per node	96 bytes	104 bytes
Build 1,000 nodes; 10% with plain handlers	99.6 µs	107.5 µs
Clone that tree	166.2 µs	163.4 µs
Render a prepared scene at 800 × 600	4.063 ms	4.080 ms

Table 1: Plain handlers compared with `main`. A microsecond (µs) is one thousandth of a millisecond (ms).

The current design adds **8 bytes per node** for the optional shared-handler slot. In this run, constructing the example tree with plain handlers took about 8% longer—roughly 8 microseconds for 1,000 nodes. The clone and rendering differences are small enough that the measurements do not establish a reliable slowdown for those operations.

For comparison, the first closure implementation used 192 bytes of handler storage per node: 96 bytes more than `main`. In an earlier run it took approximately 9.5% longer to construct a tree with plain handlers on 10% of nodes. Results from separate runs vary, so these percentages should not be treated as an exact improvement calculation.

Captured handlers still have a real cost. In a separate test of the current implementation, building a tree with captured handlers on every node took about 199 µs for 1,000 nodes, compared with about 107 µs for a tree using plain handlers on every node. That captured-state test also included cloning an `Arc` for each closure. It measured two extra allocations per captured handler: one for its node's shared handler table and one for the closure. It is a deliberately handler-heavy scenario, not a typical app benchmark.

The rendering test measures drawing an **already-built** scene. It does not include the cost of building or cloning the UI tree. Its result therefore does not mean that closures are free; it means their construction costs were outside that particular measurement.

Choosing a handler

Use a plain handler when a function can do the job without carrying app-specific values. Use a captured handler when the event needs access to state owned by the app. The current design keeps the plain case compact while allowing stateful examples and larger applications to avoid unnecessary global variables.

Prompt for reviewing or changing code

The following prompt can be copied into an AI coding assistant when working on a cssimpler application or example. Replace the final line with the files or feature you want it to review.

```
Review and update the specified cssimpler code. Keep the result
simple, preserve its behavior, and make these choices:

- Use ordinary function handlers (such as onclick) when they do
  not need to capture app-specific state.
- Use captured handlers (such as onclick_shared or
  oninput_shared) when they need access to state owned by a
  particular app instance. Share only the state they need.
- Avoid mutable globals for app or UI state. Retain globals when
  they genuinely represent shared process-wide resources, such as
  an appropriate immutable lookup table or cache.
- Avoid unnecessary allocations, reference counting, and extra
  abstractions, especially in UI construction or event handling.
  Do not trade correctness or clear code for a guessed speedup.
- Check the existing code and APIs before changing them. Test the
  behavior, including independent app instances when relevant.
  Benchmark performance-sensitive changes when practical; describe
  what was measured and any limitations instead of assuming that a
  change is faster.

Explain the changes, why each handler style was chosen, and the
checks performed.

Files or feature to review: [describe here]
```

This prompt asks for a practical balance: use the lightweight path where it fits, and pay for shared captured state where the application needs it. An optimal choice depends on the code and how often the UI is built, copied, and updated.